A Guessing Machine with a Million Dials

Explaining neural networks without the math

Wenyu (Daniel) Du · AI/ML graduate program, Indiana Wesleyan University · July 2026

The one-sentence version

Machine learning may be the discipline most fond of hiding a simple idea behind an intimidating vocabulary. “Backpropagation,” “gradient descent,” “activation functions” — these terms describe engineering details, not the essence. After building several of these systems myself, I have come to believe the essence fits in one sentence: **a neural network is a guessing machine that learns from its own mistakes.** Everything else is elaboration — so let me elaborate, carefully, one small step at a time.

What the machine actually does

Show it a photograph and it guesses “cat.” Give it half a sentence and it guesses the next word. Ask it a question about a database and it guesses the SQL query. That is genuinely all a neural network ever does: take something in, produce its best guess about what should come out. The only difference between a useless guesser and ChatGPT is how good the guesses have become.

How the guesses get good

Inside the machine are millions — in modern systems, billions — of tiny adjustable dials, which engineers call *weights*. The settings of the dials completely determine what the machine guesses, and before training begins they are set at random, so the first guesses are pure noise. Training then works like the children’s game of “hotter and colder.” We show the machine an example whose correct answer we already know. It guesses; we measure exactly how wrong the guess was; and then every dial is nudged a tiny amount in the direction that would have made the guess slightly less wrong.

A single nudge accomplishes almost nothing. Repeated millions of times across millions of examples, the nudges accumulate, and the dials drift into a configuration that guesses well — even on inputs the machine has never seen before. This nudging procedure, incidentally, is the entire meaning of the intimidating vocabulary: “gradient descent” is the nudging itself, and “backpropagation” is the bookkeeping that tells each dial which way to turn.

Two things worth emphasizing

First, nobody ever writes the rules. No line of code says that whiskers imply cat; whatever rules exist grow inside the dials, implicitly, as a side effect of being wrong millions of times. Second, the dials are arranged in layers, and the layers divide the labor. Early layers learn embarrassingly simple patterns — a bright edge in an image, or which letter tends to follow “q.” Deeper layers combine those fragments into larger ideas:

edges into whiskers, whiskers and pointed ears into “probably a cat.” What we experience as machine intelligence is simple patterns, stacked deep.

Why I trust this picture: three scales

I have watched this explanation play out at three very different scales, and the most instructive was the smallest. I built a character-level language model from scratch, working through Andrej Karpathy’s well-known “Let’s build GPT” exercise. It deliberately begins with a bigram model — a guesser so primitive it predicts each next character from the single character before it, little more than a learned frequency table. Then, step by step, you give the model the ability to attend to longer context and you stack its layers deeper, and the same learning loop — guess, measure the error, nudge the weights — turns that toy into a miniature GPT producing passable Shakespeare-flavored text. Nothing conceptually new is added along the way. The “magic” of large language models is the bigram’s guessing game at industrial scale: more dials, more context, more examples.

I saw the layered division of labor firsthand in computer vision, when I trained a YOLO object detector. The layered-dials picture predicts exactly what visualization studies of convolutional networks have documented (Zeiler & Fergus, 2014): shallow layers responding to edges and textures, deeper layers to object parts and whole objects.

And I saw the limits of the whole picture in production, when I helped my company build a text-to-SQL system over an internal database of roughly 1,500 tables. Fine-tuning on the full schema was infeasible on our hardware and timeline, so we succeeded by curating: narrowing the training data to the most relevant tables and their relationships, retrieving schema context per query with a vector database, and feeding verified correct queries back into training as new positive examples. The lesson was blunt: the dials settle into good positions only when the examples deserve to be learned from. Data quality and selection mattered far more than model size — **a neural network is only ever as smart as what it learns from.**

Why plain explanations matter for AI adoption

This exercise is more than a party trick; it sits close to the heart of leading organizations through AI change. In my experience, resistance to AI/ML integration is rarely resistance to the technology itself — it is resistance to opacity. People are asked to trust, and sometimes to be evaluated by, a system that nobody will explain to them. The moment a leader can replace “trust the black box” with “here is the guessing game happening inside,” the conversation changes character: fear gives way to informed questions about training data, failure modes, and appropriate use.

Better still, the simple model hands every stakeholder a meaningful role. If a network is only as smart as its examples, then the employees closest to the data — the people who know which records are clean, which labels are wrong, and which edge cases matter — stop being bystanders to an AI initiative and become central to it. That reframing, in my view, is one of the most underrated change-management moves available to a technical leader.

How this explanation was built

The explanation practices the communication strategies it was written to demonstrate. The content is chunked into small steps and disclosed progressively — from a one-sentence core, to mechanism, to architecture. The "hotter/colder" game and the physical dials do the work that equations would normally do. Jargon is introduced only after the plain idea it names, so vocabulary demystifies rather than intimidates. My own projects supply the relevance and the real-world evidence, and the core sentence is deliberately repeated — beginning, middle, and end — as a small act of spaced repetition. **A neural network is a guessing machine that learns from its mistakes** — and if this explanation worked, you could now explain that to anyone.

Reference

Zeiler, M. D., & Fergus, R. (2014). Visualizing and understanding convolutional networks. In *Computer Vision — ECCV 2014* (pp. 818–833). Springer.

Adapted for this portfolio from my AIML-500 Workshop 3 discussion post ("Communicating for Learning"). AI assistance (Claude, Anthropic) was used for formatting and polish; the explanation, projects, and judgments are my own.